Curs P.C.9
Metoda Greedy

① Problema rucsacului

n obiecte → $g_1, g_2, \dots, g_n$ → greutăți
→ $c_1, c_2, \dots, c_n$ → câștiguri (integrale)

G → „capacitatea" rucsacului

Câștigul maxim

variante → continuă / fracționată → se pot „tăia" → $\Theta(n \cdot \log_2 n)$
→ discretă / 0-1 → nu se pot „tăia"

⇒ toate marile sunt nr nat → $\Theta(n \cdot G)$ P.D.
→ altfel → $O(2^n)$ backtracking

Knapsack problem

$c_{ui} = \frac{c_i}{g_i}$ câștig unitar

$G = 53$ kg
$n = 7$

$$\begin{array}{cccccccc} & 1 & 2 & 3 & 4 & 5 & 6 & 7 \\ C = (& 30, & 40, & 36, & 10, & 16, & 30, & 20) \text{ Ron} \end{array}$$

$g = (10, 5, 18, 20, 8, 40, 20)$ kg

$c_u = (3, 8, 2, 0{,}5, 2, \frac{3}{4}, 1)$

ob ale	fracțiune	spațiu liber din rucsac	câștig total
2	1	$53 - 5 = 48$	40
1	1	$48 - 10 = 38$	70
5	1	$38 - 8 = 30$	86
3	1	$30 - 18 = 12$	122
7	$\frac{12}{20}$	$12 - 12 = 0$	$122 + \frac{12}{20}$

Metoda backtracking

Forma specială a pb:

Fie $A_1, A_2, \dots, A_n$ niște mulțimi, $\forall A_1, \dots, A_n \neq \emptyset$

Să se determine toate tuplele $S = (s_1, s_2, \dots, s_n)$ $\quad s_1 \in A_1,\ s_2 \in A_2,\ \dots,\ s_n \in A_n$

Ex: Să se afișeze toate nr nat formate din 5 cifre distincte și având suma cf = 30

$S = (s_1, s_2, s_3, s_4, s_5) \rightarrow$ cifre

Soluția brută:

$|A_1 \times A_2 \times \dots \times A_n| = |A_1| \cdot |A_2| \cdot \dots \cdot |A_n| \geq 2^n$ posibile soluții pt care verific proprietatea $P \Rightarrow \Theta(2^n)$

Generarea permutărilor mulțimii $A = \{1, 2, \dots, n\}$

Complex. min.: $\Theta(n!)$ $\quad n! \gg 2^n$

$n = 6$

Soluția brută

$6^6 = 46656$ tuple

$6! = 120$ permutări

$\Downarrow$

$\eta = \frac{720}{46656} \cdot 100 = 1{,}543\%$

1, 1, 1, 1, 1, 1

1, 1, 1, 1, 1, 2

⋮

* 1, 2, 3, 4, 5, 6

⋮

* 2, 1, 5, 4, 3, 6

⋮

6, 6, 6, 6, 6, 6

$n = 6$

Backtracking:

alpoll: 1, ~~1~~, ~~1~~ ~~1~~ ~~1~~ ~~1~~, 1 ⇒ 1, 2, 3, 4, 5, 6

2 ~~2~~ ~~2~~ ~~2~~ ~~2~~, 2 ⇒ 1, 2, 3, 4, 6, 5

3 3 ~~3~~ ~~3~~ ~~3~~

4 ~~4~~ ~~4~~ 4

5 ~~5~~ 5

6 6

$s_1, s_2, \ldots, s_k, s_{k+1}$

Schema generală a alg. backtracking

```
void bkt (int k){ → indice componentă curentă s_k
    int v;                                  Variabile globale
    for (v = mink; v <= maxk; v++){         int s (1005);
        s[k] = v;                                  ①
        if (s(1), .., s(k) este sol parțial){ → ②
            if (s(1), .., s(k) este soluție){
                afișez soluție;                    ③
            }
            else
                bkt (k+1);
        }
    }
}
```

Generarea permutărilor:

1) $mink = 1$, $maxk = n$

2) $s[k] \neq s(i)$, pt $1 \leq i \leq k-1$

int sol(int k){

```
int i;
for (i=1; i<k; i++)
    if (s[i]==s[k]) return 0;
return 1;
}
```

3) $k==n$